DAY 49

Deploying Machine Learning Models with Docker

A practical guide to packaging a Flask ML API into a portable container

60-Day Data Science Learning Journey

Learning outcome: train a model, package its API and dependencies, build a Docker image, run a container, and test the prediction endpoint.

1. Docker explained

Docker packages an application and its required environment into an image. You can run that image as a container on a computer with Docker. This makes deployment more repeatable across development, testing, and production environments.

Docker does not improve model accuracy by itself. It helps package the code, runtime, dependencies, and model files in a consistent way.

2. Important terminology

Dockerfile: Text instructions used to build an image.

Image: Packaged, read-only template used to create containers.

Container: An instance of an image, running or stopped.

Docker Engine: The service that builds and runs containers.

Docker Hub: A registry for storing and sharing images.

Port mapping: Connects a port on your computer to a port in the container.

Volume: Persists or shares data outside a container's writable layer.

3. Why Docker is useful for ML

ML projects rely on Python and library versions such as NumPy, scikit-learn, Flask, and joblib. Differences between computers can cause import errors or inconsistent setup. Docker lets you define the runtime environment alongside your application.

4. Project architecture

```
API client
   |
   | HTTP request to localhost:5000
   v
Host port 5000
   |
   | Docker mapping: -p 5000:5000
   v
Container port 5000
   |
   v
Flask API -> saved scikit-learn model -> JSON prediction
```

5. Generate the practical project

The supplied Python file is a project generator. Run it once. It creates a docker_ml_api folder with app.py, train_model.py, requirements.txt, Dockerfile, .dockerignore, and README.txt.

```
python Day-49-Docker-ML-Project-Generator.py
cd docker_ml_api
```

What each file does

app.py loads the saved model and exposes the API. train_model.py trains a RandomForestRegressor and saves it using joblib. requirements.txt lists dependencies. Dockerfile defines the image build. .dockerignore excludes unnecessary files. README.txt contains commands.

6. Build and run

Build the image

```
docker build -t diabetes-ml-api .
```

Docker reads the Dockerfile, installs the packages, copies the source files, trains the educational model during image build, and sets the container startup command.

Run the container

```
docker run --name diabetes-api -p 5000:5000 diabetes-ml-api
```

The first 5000 is the host port. The second 5000 is the container port. Flask binds to 0.0.0.0 so Docker can route traffic to it.

Open the home endpoint

```
http://localhost:5000/
```

Send a prediction request

```
curl -X POST http://localhost:5000/predict ^
  -H "Content-Type: application/json" ^
  -d "{\"features\":[0.038,0.05,0.06,0.02,-0.04,-0.03,-0.04,-0.003,-0.01,0.02]}"
```

The API expects a JSON object with a features list containing exactly 10 numeric values. It checks the input and returns the model prediction as JSON.

7. Understand the Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python train_model.py
EXPOSE 5000
CMD ["python", "app.py"]
```

FROM: Selects the base image.

ENV: Sets environment variables.

WORKDIR: Sets the working directory inside the image.

COPY: Copies files into the image.

RUN: Executes a command while building the image.

EXPOSE: Documents the container port. It does not publish it.

CMD: Sets the default command when the container starts.

8. Essential Docker commands

docker images , List local images.

docker ps , List running containers.

docker ps -a , List all containers.

docker logs diabetes-api , View container logs.

docker stop diabetes-api , Stop the container.

docker start diabetes-api , Start a stopped container.

docker rm -f diabetes-api , Remove the container.

docker rmi diabetes-ml-api , Remove the image when unused.

9. Common problems

Docker daemon error: Start Docker Desktop and wait for the engine to become ready.

Port 5000 is busy: Run docker run --name diabetes-api -p 5001:5000 diabetes-ml-api and open localhost:5001.

Container exits: Check docker logs diabetes-api for the startup error.

API is unreachable: Check the container status, port mapping, and Flask host setting. Use 0.0.0.0 inside the container.

10. API input validation

The endpoint checks that the request is JSON, contains a features list, has exactly 10 entries, and that every entry is numeric and finite. Validation prevents malformed values from reaching the model.

```
{
  "features": [0.038, 0.05, 0.06, 0.02, -0.04,
               -0.03, -0.04, -0.003, -0.01, 0.02]
}
```

11. Practical exercises and solutions

Task 1: Check the Docker version. **Solution:** docker --version.

Task 2: Use host port 5001. **Solution:** docker run --name diabetes-api -p 5001:5000 diabetes-ml-api, then open localhost:5001.

Task 3: View logs. **Solution:** docker logs diabetes-api.

Task 4: Explain image versus container. **Solution:** An image is the template. A container is an instance created from it.

Task 5: Explain 0.0.0.0. **Solution:** It makes the Flask process listen on container network interfaces so routed traffic can reach it.

12. Knowledge check

1. Which file contains image build instructions?

2. What is the difference between an image and a container?

3. What does -p 5000:5000 do?

4. Does EXPOSE publish a port to your computer automatically?

5. Why should Flask bind to 0.0.0.0 in this container?

6. Which command displays the container logs?

7. How many input features does the sample API expect?

Answer key

1. Dockerfile. 2. Image is a template, container is an instance. 3. Maps host port 5000 to container port 5000. 4. No, use -p. 5. So routed network traffic can reach Flask. 6. docker logs diabetes-api. 7. Ten.

13. Production improvement checklist

For production, replace Flask's development server with a production WSGI server such as Gunicorn. Pin and update dependencies, scan images for vulnerabilities, keep secrets outside the image, configure health checks, consider non-root execution, and set resource limits.

14. Dataset limitation

The sample uses scikit-learn's diabetes regression dataset to demonstrate model deployment. Its prediction is not a diagnosis, an individual's medical risk estimate, or treatment advice. Never use this educational model for clinical decisions.

15. References

Docker documentation: https://docs.docker.com/

Dockerfile reference: https://docs.docker.com/reference/dockerfile/

Docker run reference: https://docs.docker.com/reference/cli/docker/container/run/

Flask documentation: https://flask.palletsprojects.com/

scikit-learn documentation: https://scikit-learn.org/stable/